

Extra Concepts, MCQs & Coding Challenges

Table of Contents

- [Part 1: Tricky JavaScript Concepts](#)
 - [Part 2: Additional Node/Express/MongoDB Concepts](#)
 - [Part 3: Additional React Concepts](#)
 - [Part 4: Extra MCQs \(50 Questions\)](#)
 - [Part 5: More Coding Challenges \(15 Problems\)](#)
 - [Answers](#)
-

Part 1: Tricky JavaScript Concepts

1.1 Debouncing

Definition: Debouncing delays the execution of a function until a specified time has passed since the last call. Useful for search inputs, resize events - when you want to wait for the user to "stop" before executing.

```
function debounce(fn, delay) {
  let timeoutId;
  return function(...args) {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => {
      fn.apply(this, args);
    }, delay);
  };
}

// Usage: Search input
const searchInput = document.getElementById('search');
const debouncedSearch = debounce((query) => {
  console.log('Searching:', query);
  // API call here
}, 300);

searchInput.addEventListener('input', (e) => {
  debouncedSearch(e.target.value);
});

// React Hook version
function useDebounce(value, delay) {
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    const timer = setTimeout(() => setDebouncedValue(value), delay);
    return () => clearTimeout(timer);
  }, [value, delay]);
}
```

```
    return debouncedValue;
}
```

1.2 Throttling

Definition: Throttling limits function execution to once per specified time period, regardless of how many times it's called. Useful for scroll events, button clicks - when you want consistent execution rate.

```
function throttle(fn, limit) {
  let inThrottle = false;
  return function(...args) {
    if (!inThrottle) {
      fn.apply(this, args);
      inThrottle = true;
      setTimeout(() => inThrottle = false, limit);
    }
  };
}

// Usage: Scroll event
const throttledScroll = throttle(() => {
  console.log('Scroll position:', window.scrollY);
}, 100);

window.addEventListener('scroll', throttledScroll);

// Difference from debounce:
// Debounce: Waits until user STOPS, then executes once
// Throttle: Executes at regular intervals while user is active
```

1.3 Optional Chaining (?.)

Definition: Optional chaining allows reading deeply nested properties without checking each level. Returns `undefined` if any part is null/undefined instead of throwing an error.

```
const user = {
  name: 'John',
  address: {
    city: 'NYC'
  }
};

// Without optional chaining
const zip = user && user.address && user.address.zip;

// With optional chaining
const zip = user?.address?.zip; // undefined (no error)

// With function calls
```

```

const result = user.getData?.(); // undefined if getData doesn't exist

// With arrays
const first = arr?.[0];

// With nullish coalescing
const city = user?.address?.city ?? 'Unknown';

// Common patterns
const length = arr?.length ?? 0;
const name = obj?.user?.profile?.name ?? 'Guest';

```

1.4 Nullish Coalescing (??)

Definition: Returns the right-hand operand when the left is `null` or `undefined` (not other falsy values like 0, "", false). Different from `||` which returns right for ANY falsy value.

```

// || returns right for ANY falsy value
0 || 'default' // 'default' (0 is falsy)
'' || 'default' // 'default' ('' is falsy)
false || 'default' // 'default'

// ?? returns right ONLY for null/undefined
0 ?? 'default' // 0
'' ?? 'default' // ''
false ?? 'default' // false
null ?? 'default' // 'default'
undefined ?? 'default' // 'default'

// Practical example
function getConfig(options) {
  const timeout = options.timeout ?? 3000; // 0 is valid
  const retries = options.retries ?? 3; // 0 is valid
  const name = options.name || 'default'; // '' should be default
}

getConfig({ timeout: 0, retries: 0 });
// timeout = 0, retries = 0 (with ??)
// timeout = 3000, retries = 3 (with ||) ❌ Wrong!

```

1.5 Short-Circuit Evaluation

Definition: Logical operators `&&` and `||` stop evaluating as soon as the result is determined. Used for conditional execution and default values.

```

// && - Returns first falsy or last truthy
true && 'hello' // 'hello'
false && 'hello' // false
'a' && 'b' && 'c' // 'c'

```

```

'a' && '' && 'c'      // ''

// || - Returns first truthy or last falsy
false || 'hello'      // 'hello'
'a' || 'b'            // 'a'
'' || null || 'c'     // 'c'
'' || null || 0       // 0

// Practical uses
const display = user && user.name;      // Safe access
const name = userName || 'Guest';      // Default value
isLoggedIn && showDashboard();          // Conditional execution
const port = process.env.PORT || 3000; // Default config

```

1.6 Truthy and Falsy Values

Definition: In boolean context, JavaScript coerces values to true (truthy) or false (falsy). Falsy values: `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN`. Everything else is truthy.

```

// Falsy values (6 + BigInt zero)
Boolean(false)    // false
Boolean(0)        // false
Boolean(-0)       // false
Boolean(0n)       // false (BigInt zero)
Boolean("")       // false
Boolean(null)     // false
Boolean(undefined) // false
Boolean(NaN)      // false

// Truthy (everything else!)
Boolean("0")      // true (non-empty string)
Boolean("false")  // true (non-empty string)
Boolean([])       // true (empty array)
Boolean({})       // true (empty object)
Boolean(function(){} ) // true
Boolean(-1)       // true

// Common gotcha
if ([]) console.log('runs!'); // ✅ Arrays are truthy
if ({}) console.log('runs!'); // ✅ Objects are truthy
[] == false // true (coercion!)
{} == false // false

```

1.7 Type Coercion

Definition: JavaScript automatically converts types when comparing or operating on different types. `==` does coercion, `===` doesn't.

```

// String + Number
'5' + 3          // '53' (number to string)
'5' - 3          // 2 (string to number)
'5' * '2'        // 10
'foo' * 2        // NaN

// Comparison coercion
'5' == 5          // true (coerced)
'5' === 5         // false (different types)
null == undefined // true
null === undefined // false

// Array/Object coercion
[] == ''          // true ([] → '' → true)
[] == 0           // true
[1] == 1          // true
[1,2] == '1,2'    // true

// Boolean coercion
'0' == false      // true ('0' → 0 → false)
'0' === false     // false

// Best practice: Always use ===

```

1.8 Shallow vs Deep Copy

Definition: Shallow copy copies only the first level; nested objects still reference the original. Deep copy creates completely independent copies of all nested levels.

```

const original = {
  name: 'John',
  address: { city: 'NYC' },
  hobbies: ['reading']
};

// SHALLOW COPY (nested objects still linked)
const shallow1 = { ...original };
const shallow2 = Object.assign({}, original);

shallow1.name = 'Jane';           // ✅ Independent
shallow1.address.city = 'LA';     // ❌ Changes original!
console.log(original.address.city); // 'LA'

// DEEP COPY methods
// Method 1: JSON (loses functions, dates, undefined)
const deep1 = JSON.parse(JSON.stringify(original));

// Method 2: structuredClone (modern, best)
const deep2 = structuredClone(original);

```

```
// Method 3: Recursive function
function deepCopy(obj) {
  if (obj === null || typeof obj !== 'object') return obj;
  if (Array.isArray(obj)) return obj.map(item => deepCopy(item));

  const copy = {};
  for (const key in obj) {
    copy[key] = deepCopy(obj[key]);
  }
  return copy;
}

// Method 4: Lodash
const deep3 = _.cloneDeep(original);
```

1.9 call, apply, bind

Definition: Methods to explicitly set `this` context. `call` and `apply` invoke immediately with different argument formats. `bind` returns a new function with bound `this`.

```
const person = {
  name: 'John',
  greet: function(greeting, punctuation) {
    console.log(`${greeting}, I'm ${this.name}${punctuation}`);
  }
};

const anotherPerson = { name: 'Jane' };

// call - comma separated arguments
person.greet.call(anotherPerson, 'Hi', '!');
// "Hi, I'm Jane!"

// apply - array of arguments
person.greet.apply(anotherPerson, ['Hello', '?']);
// "Hello, I'm Jane?"

// bind - returns new function (doesn't execute)
const greetJane = person.greet.bind(anotherPerson);
greetJane('Hey', '...');
// "Hey, I'm Jane..."

// bind with preset arguments (partial application)
const sayHiToJane = person.greet.bind(anotherPerson, 'Hi');
sayHiToJane('!');
// "Hi, I'm Jane!"

// Common use: borrowing methods
const numbers = [1, 2, 3, 4, 5];
```

```
const max = Math.max.apply(null, numbers); // 5
// Modern: Math.max(...numbers)
```

1.10 IIFE (Immediately Invoked Function Expression)

Definition: A function that runs immediately after it's defined. Creates a private scope, useful for avoiding global variable pollution.

```
// Basic IIFE
(function() {
  const private = 'I am private';
  console.log('IIFE executed');
})();

// With parameters
(function(name) {
  console.log('Hello', name);
})('John');

// Arrow function IIFE
(() => {
  console.log('Arrow IIFE');
})();

// Returning value
const result = (function() {
  return 'value';
})();

// Module pattern with IIFE
const counter = (function() {
  let count = 0; // Private

  return {
    increment: () => ++count,
    decrement: () => --count,
    getCount: () => count
  };
})();

counter.increment(); // 1
counter.getCount(); // 1
// count is not accessible directly
```

1.11 Currying

Definition: Transforming a function with multiple arguments into a sequence of functions each taking a single argument. Enables partial application and function composition.

```

// Normal function
function add(a, b, c) {
    return a + b + c;
}
add(1, 2, 3); // 6

// Curried version
function curriedAdd(a) {
    return function(b) {
        return function(c) {
            return a + b + c;
        };
    };
}
curriedAdd(1)(2)(3); // 6

// Arrow function currying
const curriedAdd = a => b => c => a + b + c;

// Practical example: Creating specialized functions
const multiply = a => b => a * b;
const double = multiply(2);
const triple = multiply(3);

double(5); // 10
triple(5); // 15

// Generic curry function
function curry(fn) {
    return function curried(...args) {
        if (args.length >= fn.length) {
            return fn.apply(this, args);
        }
        return function(...moreArgs) {
            return curried.apply(this, args.concat(moreArgs));
        };
    };
}

const curriedSum = curry((a, b, c) => a + b + c);
curriedSum(1)(2)(3); // 6
curriedSum(1, 2)(3); // 6
curriedSum(1)(2, 3); // 6

```

1.12 Memoization

Definition: Caching the results of expensive function calls and returning the cached result when the same inputs occur again. Trades memory for speed.

```
// Simple memoization
function memoize(fn) {
  const cache = {};
  return function(...args) {
    const key = JSON.stringify(args);
    if (key in cache) {
      console.log('From cache');
      return cache[key];
    }
    const result = fn.apply(this, args);
    cache[key] = result;
    return result;
  };
}

// Example: Fibonacci with memoization
const fib = memoize(function(n) {
  if (n <= 1) return n;
  return fib(n - 1) + fib(n - 2);
});

fib(40); // Fast!

// React useMemo is a form of memoization
const expensiveValue = useMemo(() => {
  return computeExpensiveValue(a, b);
}, [a, b]);
```

1.13 Event Delegation

Definition: Instead of attaching event listeners to each child element, attach one listener to a parent that catches bubbling events. More efficient and works with dynamically added elements.

```
// ❌ Bad: Listener on each item
document.querySelectorAll('.item').forEach(item => {
  item.addEventListener('click', handleClick);
});

// ✅ Good: Event delegation on parent
document.querySelector('.list').addEventListener('click', (e) => {
  if (e.target.classList.contains('item')) {
    handleClick(e);
  }
  // Or check tag
  if (e.target.tagName === 'LI') {
    console.log('List item clicked:', e.target.textContent);
  }
});

// Works with dynamically added elements automatically!
```

```
// e.target = element that was clicked
// e.currentTarget = element with the listener
```

1.14 Object.freeze vs Object.seal

Definition: `freeze` prevents all modifications (add, remove, modify). `seal` prevents adding/removing but allows modifying existing properties.

```
// Object.freeze - Complete immutability (shallow)
const frozen = Object.freeze({ a: 1, b: { c: 2 } });
frozen.a = 10;      // ❌ Silently fails
frozen.d = 4;       // ❌ Can't add
delete frozen.a;    // ❌ Can't delete
frozen.b.c = 20;    // ✅ Nested objects not frozen!

Object.isFrozen(frozen); // true

// Object.seal - No add/delete, can modify
const sealed = Object.seal({ a: 1 });
sealed.a = 10;      // ✅ Can modify
sealed.b = 2;       // ❌ Can't add
delete sealed.a;    // ❌ Can't delete

Object.isSealed(sealed); // true

// Deep freeze
function deepFreeze(obj) {
  Object.keys(obj).forEach(key => {
    if (typeof obj[key] === 'object' && obj[key] !== null) {
      deepFreeze(obj[key]);
    }
  });
  return Object.freeze(obj);
}
```

1.15 Generator Functions

Definition: Functions that can be paused and resumed, yielding multiple values over time. Created with `function*` and use `yield` keyword.

```
function* numberGenerator() {
  yield 1;
  yield 2;
  yield 3;
}

const gen = numberGenerator();
gen.next(); // { value: 1, done: false }
gen.next(); // { value: 2, done: false }
```

```

gen.next(); // { value: 3, done: false }
gen.next(); // { value: undefined, done: true }

// Infinite sequence
function* idGenerator() {
  let id = 1;
  while (true) {
    yield id++;
  }
}

const ids = idGenerator();
ids.next().value; // 1
ids.next().value; // 2

// Iterable with for...of
function* range(start, end) {
  for (let i = start; i <= end; i++) {
    yield i;
  }
}

for (const num of range(1, 5)) {
  console.log(num); // 1, 2, 3, 4, 5
}

```

1.16 Symbol

Definition: A primitive type that creates unique, immutable identifiers. Useful for object property keys that won't collide with other properties.

```

// Create unique symbols
const sym1 = Symbol('description');
const sym2 = Symbol('description');
sym1 === sym2; // false (always unique)

// Use as object key
const ID = Symbol('id');
const user = {
  name: 'John',
  [ID]: 12345
};

user[ID]; // 12345
Object.keys(user); // ['name'] - Symbol not included!

// Well-known symbols
Symbol.iterator // Make object iterable
Symbol.toStringTag // Customize Object.prototype.toString

```

```
// Symbol.iterator example
const range = {
  from: 1,
  to: 5,
  [Symbol.iterator]() {
    let current = this.from;
    const last = this.to;
    return {
      next() {
        if (current <= last) {
          return { value: current++, done: false };
        }
        return { done: true };
      }
    };
  }
};

[...range]; // [1, 2, 3, 4, 5]
```

1.17 WeakMap and WeakSet

Definition: Collections that hold "weak" references to objects. Keys (WeakMap) or values (WeakSet) can be garbage collected when no other references exist. Not iterable.

```
// WeakMap - keys must be objects
const wm = new WeakMap();
let obj = { name: 'John' };

wm.set(obj, 'metadata');
wm.get(obj); // 'metadata'

obj = null; // Object can be garbage collected
// Entry in WeakMap is automatically removed

// Use case: Private data
const privateData = new WeakMap();

class User {
  constructor(name) {
    privateData.set(this, { name });
  }
  getName() {
    return privateData.get(this).name;
  }
}

// WeakSet - values must be objects
const ws = new WeakSet();
ws.add(obj);
```

```

ws.has(obj); // true

// Use case: Track visited/processed objects
const visited = new WeakSet();
function processOnce(obj) {
  if (visited.has(obj)) return;
  visited.add(obj);
  // Process...
}

```

1.18 Proxy and Reflect

Definition: Proxy wraps an object and intercepts operations like get, set, delete. Reflect provides methods that mirror Proxy traps for proper forwarding.

```

const user = { name: 'John', age: 30 };

const proxy = new Proxy(user, {
  get(target, prop) {
    console.log(`Getting ${prop}`);
    return Reflect.get(target, prop);
  },
  set(target, prop, value) {
    console.log(`Setting ${prop} to ${value}`);
    if (prop === 'age' && typeof value !== 'number') {
      throw new Error('Age must be a number');
    }
    return Reflect.set(target, prop, value);
  }
});

proxy.name; // Logs: "Getting name", returns "John"
proxy.age = 31; // Logs: "Setting age to 31"
proxy.age = 'old'; // Throws error

// Reactive data (Vue.js style)
function reactive(obj) {
  return new Proxy(obj, {
    set(target, prop, value) {
      const result = Reflect.set(target, prop, value);
      console.log('Data changed, update UI');
      return result;
    }
  });
}

```

Part 2: Additional Node/Express/MongoDB Concepts

2.1 CORS (Cross-Origin Resource Sharing)

Definition: Security mechanism that restricts HTTP requests from one origin to another. Browsers block cross-origin requests by default; server must send CORS headers to allow it.

```
// Express CORS setup
const cors = require('cors');

// Allow all origins
app.use(cors());

// Specific configuration
app.use(cors({
  origin: 'http://localhost:3000',
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization'],
  credentials: true
}));

// Multiple origins
const allowedOrigins = ['http://localhost:3000', 'https://myapp.com'];
app.use(cors({
  origin: (origin, callback) => {
    if (!origin || allowedOrigins.includes(origin)) {
      callback(null, true);
    } else {
      callback(new Error('Not allowed by CORS'));
    }
  }
}));
```

2.2 Rate Limiting

Definition: Limiting the number of requests a client can make in a time period. Prevents abuse, DDoS attacks, and API overuse.

```
const rateLimit = require('express-rate-limit');

const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100, // 100 requests per window
  message: 'Too many requests, please try again later',
  standardHeaders: true,
  legacyHeaders: false
});

app.use('/api', limiter);

// Different limits for different routes
```

```
const authLimiter = ratelimit({
  windowMs: 60 * 60 * 1000, // 1 hour
  max: 5, // 5 login attempts per hour
  message: 'Too many login attempts'
});

app.use('/api/auth/login', authLimiter);
```

2.3 MongoDB Transactions

Definition: Group multiple operations into atomic units - either all succeed or all fail. Essential for data consistency across multiple documents.

```
const session = await mongoose.startSession();

try {
  session.startTransaction();

  // All operations use the session
  await Account.updateOne(
    { _id: fromAccountId },
    { $inc: { balance: -amount } },
    { session }
  );

  await Account.updateOne(
    { _id: toAccountId },
    { $inc: { balance: amount } },
    { session }
  );

  await session.commitTransaction();
  console.log('Transfer successful');
} catch (error) {
  await session.abortTransaction();
  console.log('Transfer failed, rolled back');
} finally {
  session.endSession();
}
```

2.4 Helmet (Security Headers)

Definition: Express middleware that sets various HTTP headers to protect against common web vulnerabilities like XSS, clickjacking, etc.

```
const helmet = require('helmet');

app.use(helmet()); // Enables all default protections
```

```
// Specific options
app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ['self'],
      scriptSrc: ['self', "trusted-cdn.com"]
    }
  },
  crossOriginEmbedderPolicy: false
}));

// What helmet sets:
// X-Content-Type-Options: nosniff
// X-Frame-Options: DENY
// X-XSS-Protection: 0
// Strict-Transport-Security
// And more...
```

Part 3: Additional React Concepts

3.1 React.memo

Definition: Higher-order component that memoizes a component, preventing re-renders if props haven't changed. For functional components, like `PureComponent` for classes.

```
const ExpensiveComponent = React.memo(function ExpensiveComponent({ data }) {
  console.log('Rendering...');
  return <div>{data.name}</div>;
});

// With custom comparison
const MyComponent = React.memo(
  function MyComponent({ user }) {
    return <div>{user.name}</div>;
  },
  (prevProps, nextProps) => {
    // Return true to skip re-render
    return prevProps.user.id === nextProps.user.id;
  }
);
```

3.2 forwardRef

Definition: Passes a `ref` through a component to one of its children. Needed when parent needs direct access to a child's DOM element.

```
const FancyInput = React.forwardRef((props, ref) => {
  return <input ref={ref} className="fancy" {...props} />;
});
```

```

});

function Parent() {
  const inputRef = useRef();

  const focusInput = () => {
    inputRef.current.focus();
  };

  return (
    <>
      <FancyInput ref={inputRef} />
      <button onClick={focusInput}>Focus</button>
    </>
  );
}

```

3.3 useImperativeHandle

Definition: Customizes the value exposed to parent when using `ref` with `forwardRef`. Lets you expose specific methods instead of the DOM element.

```

const FancyInput = React.forwardRef((props, ref) => {
  const inputRef = useRef();

  useImperativeHandle(ref, () => ({
    focus: () => inputRef.current.focus(),
    clear: () => inputRef.current.value = '',
    getValue: () => inputRef.current.value
  }));

  return <input ref={inputRef} {...props} />;
});

function Parent() {
  const inputRef = useRef();

  return (
    <>
      <FancyInput ref={inputRef} />
      <button onClick={() => inputRef.current.focus()}>Focus</button>
      <button onClick={() => inputRef.current.clear()}>Clear</button>
    </>
  );
}

```

3.4 Error Boundaries

Definition: Components that catch JavaScript errors in their child component tree, log errors, and display fallback UI. Only class components can be error boundaries.

```
class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false };
  }

  static getDerivedStateFromError(error) {
    return { hasError: true };
  }

  componentDidCatch(error, errorInfo) {
    console.error('Error:', error);
    console.error('Info:', errorInfo);
    // Log to error reporting service
  }

  render() {
    if (this.state.hasError) {
      return <h1>Something went wrong.</h1>;
    }
    return this.props.children;
  }
}

// Usage
<ErrorBoundary>
  <MyComponent />
</ErrorBoundary>
```

3.5 Lazy Loading & Suspense

Definition: `React.Lazy` dynamically imports components (code splitting). `Suspense` shows fallback while lazy component loads.

```
const LazyComponent = React.lazy(() => import('./HeavyComponent'));

function App() {
  return (
    <Suspense fallback={<div>Loading...</div>}>
      <LazyComponent />
    </Suspense>
  );
}

// Route-based splitting
const Home = lazy(() => import('./pages/Home'));
const About = lazy(() => import('./pages/About'));
```

```
function App() {
  return (
    <Suspense fallback=<Loading />>
      <Routes>
        <Route path="/" element=<Home /> />
        <Route path="/about" element=<About /> />
      </Routes>
    </Suspense>
  );
}
```

3.6 useLayoutEffect

Definition: Same as `useEffect` but fires synchronously after DOM mutations, before browser paint. Use for DOM measurements or preventing visual flicker.

```
// useEffect: Runs AFTER paint (async)
// useLayoutEffect: Runs BEFORE paint (sync)

function Tooltip({ children, targetRef }) {
  const [position, setPosition] = useState({ top: 0, left: 0 });

  // Use useLayoutEffect to measure and position BEFORE paint
  useLayoutEffect(() => {
    const rect = targetRef.current.getBoundingClientRect();
    setPosition({
      top: rect.bottom,
      left: rect.left
    });
  }, [targetRef]);

  return (
    <div style={{ position: 'absolute', ...position }}>
      {children}
    </div>
  );
}

// Rule: Start with useEffect, switch to useLayoutEffect if you see flicker
```

3.7 Portals

Definition: Render children into a DOM node outside the parent component's hierarchy. Useful for modals, tooltips, overlays.

```
import { createPortal } from 'react-dom';

function Modal({ children, onClose }) {
```

```

    return createPortal(
      <div className="modal-overlay" onClick={onClose}>
        <div className="modal" onClick={e => e.stopPropagation()}>
          {children}
        </div>
      </div>,
      document.getElementById('modal-root')
    );
  }

// In index.html: <div id="modal-root"></div>

// Usage
function App() {
  const [showModal, setShowModal] = useState(false);

  return (
    <div>
      <button onClick={() => setShowModal(true)}>Open</button>
      {showModal && (
        <Modal onClose={() => setShowModal(false)}>
          <h2>Modal Content</h2>
        </Modal>
      )}
    </div>
  );
}

```

Part 4: Extra MCQs (50 Questions)

Q1. What does optional chaining return for undefined property?

```

const obj = {};
console.log(obj?.a?.b);

```

- A) Error
- B) null
- C) undefined
- D) 0

Q2. What's the difference between ?? and || ?

- A) No difference
- B) ?? only checks for null/undefined, || checks all falsy
- C) || only checks for null/undefined
- D) ?? is faster

Q3. What is the output?

```
console.log(0 || 'default');  
console.log(0 ?? 'default');
```

- A) 'default', 'default'
 - B) 0, 0
 - C) 'default', 0
 - D) 0, 'default'
-

Q4. What is debouncing used for?

- A) Making functions run faster
 - B) Delaying execution until input stops
 - C) Running functions at intervals
 - D) Caching results
-

Q5. What is throttling used for?

- A) Limiting execution to once per time period
 - B) Speeding up functions
 - C) Pausing functions
 - D) Caching results
-

Q6. What is the output?

```
console.log([] == false);  
console.log([] === false);  
console.log(![] === true);
```

- A) true, false, true
 - B) true, true, true
 - C) false, false, false
 - D) true, false, false
-

Q7. What is a shallow copy?

- A) Copy of only primitive values
 - B) Copy where nested objects share references
 - C) Complete independent copy
 - D) Copy of only first property
-

Q8. What does `Object.freeze()` do?

- A) Prevents adding properties only
 - B) Prevents all modifications
 - C) Deep freezes the object
 - D) Deletes all properties
-

Q9. What is an IIFE?

- A) A function that runs immediately after definition
 - B) A type of loop
 - C) An async function
 - D) A class method
-

Q10. What is currying?

- A) Adding spices to code
 - B) Converting multi-arg function to chain of single-arg functions
 - C) Caching function results
 - D) Running functions in parallel
-

Q11. What is memoization?

- A) Writing code from memory
 - B) Caching function results for the same inputs
 - C) Memory management technique
 - D) Converting to memos
-

Q12. What is event delegation?

- A) Delegating events to other developers
 - B) Attaching listener to parent instead of each child
 - C) Creating custom events
 - D) Preventing default events
-

Q13. Map vs Object - which is correct?

- A) Map keys can be any type, Object keys are strings
 - B) Object keys can be any type
 - C) No difference
 - D) Map is slower
-

Q14. What is a WeakMap?

- A) A map with weak performance
 - B) A map where keys are weakly referenced (can be garbage collected)
 - C) A map with fewer methods
 - D) An immutable map
-

Q15. What does Proxy do?

- A) Makes HTTP requests
 - B) Intercepts operations on an object
 - C) Creates a network proxy
 - D) Compresses data
-

Q16. What is CORS?

- A) A JavaScript framework
- B) Cross-Origin Resource Sharing - security for cross-domain requests

- C) A database
 - D) A CSS property
-

Q17. What does `helmet` middleware do?

- A) Adds styling
 - B) Sets security HTTP headers
 - C) Compresses responses
 - D) Parses JSON
-

Q18. What is rate limiting?

- A) Limiting code length
 - B) Limiting requests per time period
 - C) Limiting response size
 - D) Limiting database queries
-

Q19. What is `React.memo` used for?

- A) Memoizing values
 - B) Preventing unnecessary component re-renders
 - C) Creating memos
 - D) Memory management
-

Q20. What is `forwardRef` used for?

- A) Forwarding emails
 - B) Passing ref through component to child
 - C) Creating refs
 - D) Forwarding props
-

Q21. What is the output?

```
const a = { x: 1 };
const b = { x: 1 };
console.log(a === b);
console.log(a.x === b.x);
```

- A) `true, true`
 - B) `false, true`
 - C) `true, false`
 - D) `false, false`
-

Q22. What is the output?

```
console.log(typeof NaN);
console.log(NaN === NaN);
```

- A) 'NaN', true
 - B) 'number', false
 - C) 'undefined', false
 - D) 'number', true
-

Q23. Which creates a deep copy?

- A) Object.assign()
 - B) Spread operator {...obj}
 - C) JSON.parse(JSON.stringify(obj))
 - D) obj.slice()
-

Q24. What is event bubbling?

- A) Events bubble up from child to parent
 - B) Events bubble down from parent to child
 - C) Events disappear
 - D) Events repeat
-

Q25. What is the output?

```
console.log('10' - 5);  
console.log('10' + 5);
```

- A) 5, 15
 - B) 5, '105'
 - C) '105', 15
 - D) NaN, '105'
-

Q26. What does useLayoutEffect do differently than useEffect ?

- A) Runs after paint
 - B) Runs before paint, synchronously
 - C) No difference
 - D) Only runs once
-

Q27. What is a React Portal?

- A) A routing method
 - B) Rendering children outside parent DOM hierarchy
 - C) A state management tool
 - D) An animation library
-

Q28. What is code splitting?

- A) Splitting code into multiple files for lazy loading
 - B) Splitting code for different developers
 - C) Commenting code
 - D) Minifying code
-

Q29. What does `React.lazy()` do?

- A) Makes components slow
 - B) Dynamically imports components
 - C) Creates lazy state
 - D) Delays rendering
-

Q30. What is the output?

```
const arr = [1, 2, 3];  
const [a, , b] = arr;  
console.log(a, b);
```

- A) 1, 2
 - B) 1, 3
 - C) 2, 3
 - D) Error
-

Q31. What is the output?

```
let x = 1;  
let y = x++;  
console.log(x, y);
```

- A) 1, 1
 - B) 2, 1
 - C) 2, 2
 - D) 1, 2
-

Q32. What is the purpose of `Symbol` ?

- A) Mathematical operations
 - B) Creating unique identifiers
 - C) String manipulation
 - D) DOM manipulation
-

Q33. What method makes an object iterable?

- A) `Symbol.for`
 - B) `Symbol.iterator`
 - C) `Symbol.iterable`
 - D) `Object.iterate`
-

Q34. What is the output?

```
const obj = { a: 1 };  
Object.seal(obj);
```

```
obj.a = 2;  
obj.b = 3;  
console.log(obj);
```

- A) { a: 2, b: 3 }
 - B) { a: 2 }
 - C) { a: 1 }
 - D) Error
-

Q35. What's the difference between `null` and `undefined` ?

- A) No difference
 - B) `undefined` = not assigned, `null` = intentional absence
 - C) `null` = not assigned, `undefined` = intentional absence
 - D) Both are errors
-

Q36. What is the output?

```
console.log(typeof typeof 1);
```

- A) 'number'
 - B) 'string'
 - C) 'undefined'
 - D) 1
-

Q37. What does `Array.from()` do?

- A) Creates array from another array
 - B) Creates array from array-like or iterable
 - C) Removes array elements
 - D) Sorts array
-

Q38. What is the output?

```
console.log([1,2,3] + [4,5,6]);
```

- A) [1,2,3,4,5,6]
 - B) '1,2,34,5,6'
 - C) 10
 - D) Error
-

Q39. What is tree shaking?

- A) Animation effect
 - B) Dead code elimination in bundling
 - C) DOM manipulation
 - D) File organization
-

Q40. What is the output?

```
const fn = () => arguments;  
console.log(fn(1, 2, 3));
```

- A) [1, 2, 3]
 - B) Error or undefined
 - C) { 0: 1, 1: 2, 2: 3 }
 - D) 3
-

Q41. What does useImperativeHandle do?

- A) Handles forms
 - B) Customizes ref value exposed to parent
 - C) Creates imperative code
 - D) Handles errors
-

Q42. Error boundaries in React can be:

- A) Functional components only
 - B) Class components only
 - C) Both class and functional
 - D) Neither
-

Q43. What is the output?

```
console.log(1 < 2 < 3);  
console.log(3 > 2 > 1);
```

- A) true, true
 - B) true, false
 - C) false, true
 - D) false, false
-

Q44. What does Object.entries() return?

- A) Array of keys
 - B) Array of values
 - C) Array of [key, value] pairs
 - D) Object copy
-

Q45. What is the output?

```
console.log([] + {});  
console.log({} + []);
```

- A) '[object Object]', 0
 - B) '[object Object]', '[object Object]'
 - C) Both '[object Object]'
 - D) Error
-

Q46. What is the purpose of Reflect ?

- A) Mirror operations
 - B) Provide methods for Proxy traps
 - C) Create reflections
 - D) Debug code
-

Q47. MongoDB \$lookup is similar to SQL:

- A) WHERE
 - B) JOIN
 - C) GROUP BY
 - D) ORDER BY
-

Q48. What does express.json() middleware do?

- A) Sends JSON response
 - B) Parses JSON request body
 - C) Validates JSON
 - D) Compresses JSON
-

Q49. What is the output?

```
console.log('b' + 'a' + + 'a' + 'a');
```

- A) 'baaa'
 - B) 'baNaNa'
 - C) 'ba0a'
 - D) Error
-

Q50. What is useId hook used for?

- A) Creating user IDs
 - B) Generating unique IDs for accessibility
 - C) Database IDs
 - D) Component IDs
-

Part 5: More Coding Challenges (15 Problems)

Challenge 1: Implement throttle function

```
// throttle(fn, limit) - executes fn at most once per limit ms
```

Challenge 2: Deep Clone Object

```
// deepClone(obj) - create completely independent copy
```

Challenge 3: Flatten Object

```
// Input: { a: { b: { c: 1 } }, d: 2 }  
// Output: { 'a.b.c': 1, 'd': 2 }
```

Challenge 4: Compose Functions

```
// compose(f, g, h)(x) = f(g(h(x)))
```

Challenge 5: Promisify Callback Function

```
// Convert callback-style function to Promise  
// promisify(fs.readFile)(file.txt).then(...)
```

Challenge 6: LRU Cache

```
// Implement Least Recently Used cache with get and put
```

Challenge 7: Event Emitter

```
// Implement on(event, callback), emit(event, data), off(event, callback)
```

Challenge 8: Retry with Exponential Backoff

```
// retry(fn, maxRetries) - retry failed async function with increasing delays
```

Challenge 9: Rate Limiter

```
// rateLimiter(fn, limit, interval) - allow max 'limit' calls per 'interval'
```

Challenge 10: Serialize Async Functions

```
// Execute array of async functions in sequence
```

Challenge 11: Type Checker

```
// getType(value) - return accurate type for any value
// getType([]) → 'array', getType(null) → 'null', getType(/regex/) → 'regexp'
```

Challenge 12: Object Path Access

```
// get(obj, 'a.b.c', defaultValue) - safely access nested property
// set(obj, 'a.b.c', value) - set nested property
```

Challenge 13: Chunk Array

```
// chunk([1,2,3,4,5], 2) → [[1,2], [3,4], [5]]
```

Challenge 14: Intersection of Multiple Arrays

```
// intersection([1,2,3], [2,3,4], [3,4,5]) → [3]
```

Challenge 15: String Compression

```
// compress('aaabbbcc') → 'a3b3c2'
// Return original if compressed is longer
```

Answers

MCQ Answers

Q#	Ans	Explanation
1	C	Optional chaining returns undefined for missing properties
2	B	?? only for null/undefined, `
3	C	`0
4	B	Debouncing delays until activity stops
5	A	Throttling limits to once per period
6	A	Empty array is truthy but == false due to coercion
7	B	Shallow copy shares nested references

8	B	Freeze prevents all modifications (shallow)
9	A	IIFE runs immediately after definition
10	B	Currying transforms to chain of single-arg functions
11	B	Memoization caches results for same inputs
12	B	Event delegation uses parent listener
13	A	Map keys can be any type
14	B	WeakMap keys can be garbage collected
15	B	Proxy intercepts object operations
16	B	CORS is cross-origin security mechanism
17	B	Helmet sets security headers
18	B	Rate limiting restricts requests per time
19	B	React.memo prevents unnecessary re-renders
20	B	forwardRef passes ref to child
21	B	Objects compared by reference, primitives by value
22	B	NaN is type number, never equal to itself
23	C	JSON parse/stringify creates deep copy
24	A	Events bubble from child to parent
25	B	- coerces to number, + concatenates
26	B	useLayoutEffect runs synchronously before paint
27	B	Portal renders outside parent hierarchy
28	A	Code splitting for lazy loading
29	B	React.lazy dynamically imports
30	B	Skipping with comma gets 1 and 3
31	B	Post-increment returns then increments
32	B	Symbol creates unique identifiers
33	B	Symbol.iterator makes objects iterable
34	B	Sealed: can modify, can't add/remove
35	B	undefined = not assigned, null = intentional
36	B	typeof 1 = 'number', typeof 'number' = 'string'
37	B	Array.from creates from array-like/iterable

38	B	Arrays coerce to strings, then concatenate
39	B	Tree shaking removes unused code
40	B	Arrow functions don't have arguments
41	B	Customizes ref value for parent
42	B	Only class components can be error boundaries
43	B	<code>1 < 2 = true = 1, 1 < 3 = true. 3 > 2 = true = 1, 1 > 1 = false</code>
44	C	Returns [key, value] pairs
45	A	<code>[] + {} = '[object Object]', {} + []</code> treated as statement + array = 0
46	B	Reflect provides Proxy trap methods
47	B	<code>\$lookup</code> is similar to JOIN
48	B	Parses JSON request body
49	B	<code>+ 'a' = NaN</code> , so <code>'ba' + 'NaN' + 'a' = 'baNaNNa'</code>
50	B	<code>useId</code> generates unique IDs for accessibility

Coding Solutions

Solution 1: Throttle

```
function throttle(fn, limit) {
  let inThrottle = false;
  return function(...args) {
    if (!inThrottle) {
      fn.apply(this, args);
      inThrottle = true;
      setTimeout(() => inThrottle = false, limit);
    }
  };
}
```

Solution 2: Deep Clone

```
function deepClone(obj) {
  if (obj === null || typeof obj !== 'object') return obj;
  if (obj instanceof Date) return new Date(obj);
  if (obj instanceof RegExp) return new RegExp(obj);
  if (Array.isArray(obj)) return obj.map(item => deepClone(item));

  const clone = {};
  for (const key in obj) {
    if (obj.hasOwnProperty(key)) {
```

```

        clone[key] = deepClone(obj[key]);
    }
}
return clone;
}

// Modern: structuredClone(obj)

```

Solution 3: Flatten Object

```

function flattenObject(obj, prefix = '') {
    return Object.keys(obj).reduce((acc, key) => {
        const newKey = prefix ? `${prefix}.${key}` : key;
        if (typeof obj[key] === 'object' && obj[key] !== null && !Array.isArray(obj[key])) {
            Object.assign(acc, flattenObject(obj[key], newKey));
        } else {
            acc[newKey] = obj[key];
        }
        return acc;
    }, {});
}

```

Solution 4: Compose

```

const compose = (...fns) => (x) =>
    fns.reduceRight((acc, fn) => fn(acc), x);

// Usage
const add1 = x => x + 1;
const multiply2 = x => x * 2;
const subtract3 = x => x - 3;

compose(add1, multiply2, subtract3)(5); // add1(multiply2(subtract3(5))) = 5

```

Solution 5: Promisify

```

function promisify(fn) {
    return function(...args) {
        return new Promise((resolve, reject) => {
            fn(...args, (err, result) => {
                if (err) reject(err);
                else resolve(result);
            });
        });
    };
}

```

Solution 6: LRU Cache

```
class LRUCache {
  constructor(capacity) {
    this.capacity = capacity;
    this.cache = new Map();
  }

  get(key) {
    if (!this.cache.has(key)) return -1;
    const value = this.cache.get(key);
    this.cache.delete(key);
    this.cache.set(key, value);
    return value;
  }

  put(key, value) {
    if (this.cache.has(key)) this.cache.delete(key);
    else if (this.cache.size >= this.capacity) {
      this.cache.delete(this.cache.keys().next().value);
    }
    this.cache.set(key, value);
  }
}
```

Solution 7: Event Emitter

```
class EventEmitter {
  constructor() {
    this.events = {};
  }

  on(event, callback) {
    if (!this.events[event]) this.events[event] = [];
    this.events[event].push(callback);
    return this;
  }

  emit(event, ...args) {
    if (!this.events[event]) return;
    this.events[event].forEach(cb => cb(...args));
    return this;
  }

  off(event, callback) {
    if (!this.events[event]) return;
    this.events[event] = this.events[event].filter(cb => cb !== callback);
    return this;
  }
}
```

```
    }  
  }  
}
```

Solution 8: Retry with Backoff

```
async function retry(fn, maxRetries = 3, delay = 1000) {  
  for (let i = 0; i < maxRetries; i++) {  
    try {  
      return await fn();  
    } catch (error) {  
      if (i === maxRetries - 1) throw error;  
      await new Promise(r => setTimeout(r, delay * Math.pow(2, i)));  
    }  
  }  
}
```

Solution 9: Rate Limiter

```
function ratelimiter(fn, limit, interval) {  
  const queue = [];  
  let count = 0;  
  
  setInterval(() => { count = 0; }, interval);  
  
  return function(...args) {  
    if (count < limit) {  
      count++;  
      return fn.apply(this, args);  
    }  
    return new Promise(resolve => {  
      queue.push(() => resolve(fn.apply(this, args)));  
    });  
  };  
}
```

Solution 10: Serialize Async

```
async function serialize(asyncFunctions) {  
  const results = [];  
  for (const fn of asyncFunctions) {  
    results.push(await fn());  
  }  
  return results;  
}  
  
// Or with reduce  
const serialize = (fns) => fns.reduce(  
  (promise, fn) => promise.then(results =>
```

```
    fn().then(result => [...results, result])
  ),
  Promise.resolve([])
);
```

Solution 11: Type Checker

```
function getType(value) {
  if (value === null) return 'null';
  if (Array.isArray(value)) return 'array';
  if (value instanceof Date) return 'date';
  if (value instanceof RegExp) return 'regexp';
  if (value instanceof Map) return 'map';
  if (value instanceof Set) return 'set';
  return typeof value;
}
```

Solution 12: Object Path

```
function get(obj, path, defaultValue) {
  const keys = path.split('.');
  let result = obj;
  for (const key of keys) {
    result = result?.[key];
    if (result === undefined) return defaultValue;
  }
  return result;
}

function set(obj, path, value) {
  const keys = path.split('.');
  let current = obj;
  for (let i = 0; i < keys.length - 1; i++) {
    if (!(keys[i] in current)) current[keys[i]] = {};
    current = current[keys[i]];
  }
  current[keys[keys.length - 1]] = value;
  return obj;
}
```

Solution 13: Chunk Array

```
function chunk(arr, size) {
  const result = [];
  for (let i = 0; i < arr.length; i += size) {
    result.push(arr.slice(i, i + size));
  }
  return result;
}
```

```
}

// One-liner
const chunk = (arr, size) =>
  Array.from({ length: Math.ceil(arr.length / size) },
    (_, i) => arr.slice(i * size, i * size + size));
```

Solution 14: Multi-Array Intersection

```
function intersection(...arrays) {
  if (arrays.length === 0) return [];
  return arrays.reduce((acc, arr) =>
    acc.filter(item => arr.includes(item))
  );
}
```

Solution 15: String Compression

```
function compress(str) {
  let result = '';
  let count = 1;

  for (let i = 0; i < str.length; i++) {
    if (str[i] === str[i + 1]) {
      count++;
    } else {
      result += str[i] + count;
      count = 1;
    }
  }

  return result.length < str.length ? result : str;
}
```

Good luck with your interview! 🍀